

Algorithmes de tri

rouanehm@gmail.com

Plan

- Le problème de tri
- Contexte général du tri
- Les algorithmes populaires de tri
 1. Tri par sélection
 2. Tri par insertion
 3. Tri par bulles (Bubble Sort)
 4. Tri par fusion (Merge Sort)
 5. Tri insertion binaire (Binary Insertion Sort)

Le problème de tri

- Un des problèmes algorithmiques les plus fondamentaux
- Input : une séquence de n nombres $e_1, e_2, \dots, e_n$ munie d'une relation d'ordre total, p.e., $\leq$
- Output : une permutation de la séquence de départ tel que un ordre total est établi : $e'_1 \leq e'_2 \leq \dots \leq e'_n$
- Applications :
 - Tri des résultats d'un moteur de recherche
 - Tri des messages selon un critère (date, priorité, etc.)
 - Des études montrent que 25% du temps du CPU est utilisé pour faire le tri
- Pour le reste du cours, on suppose pour simplifier qu'on trie des tableaux d'entiers car souvent on a trier des clés de tout types.

Contexte général du tri

- On dispose de n données en mémoire centrale (tri interne, par opposition au tri externe d'une séquence dans une mémoire de masse) qu'il s'agit d'ordonner d'une certaine manière (ex. croissante).
- Les données sont comparables (relation d'ordre entre les items)
- deux classes d'algorithmes :
 - en temps $O(n^2)$ dans le cas le pire
 - en temps $O(n \log n)$ dans le cas le pire
- La complexité minimale possible est en $O(n \log n)$

Tri par sélection (Selection Sort)

Principe et complexité

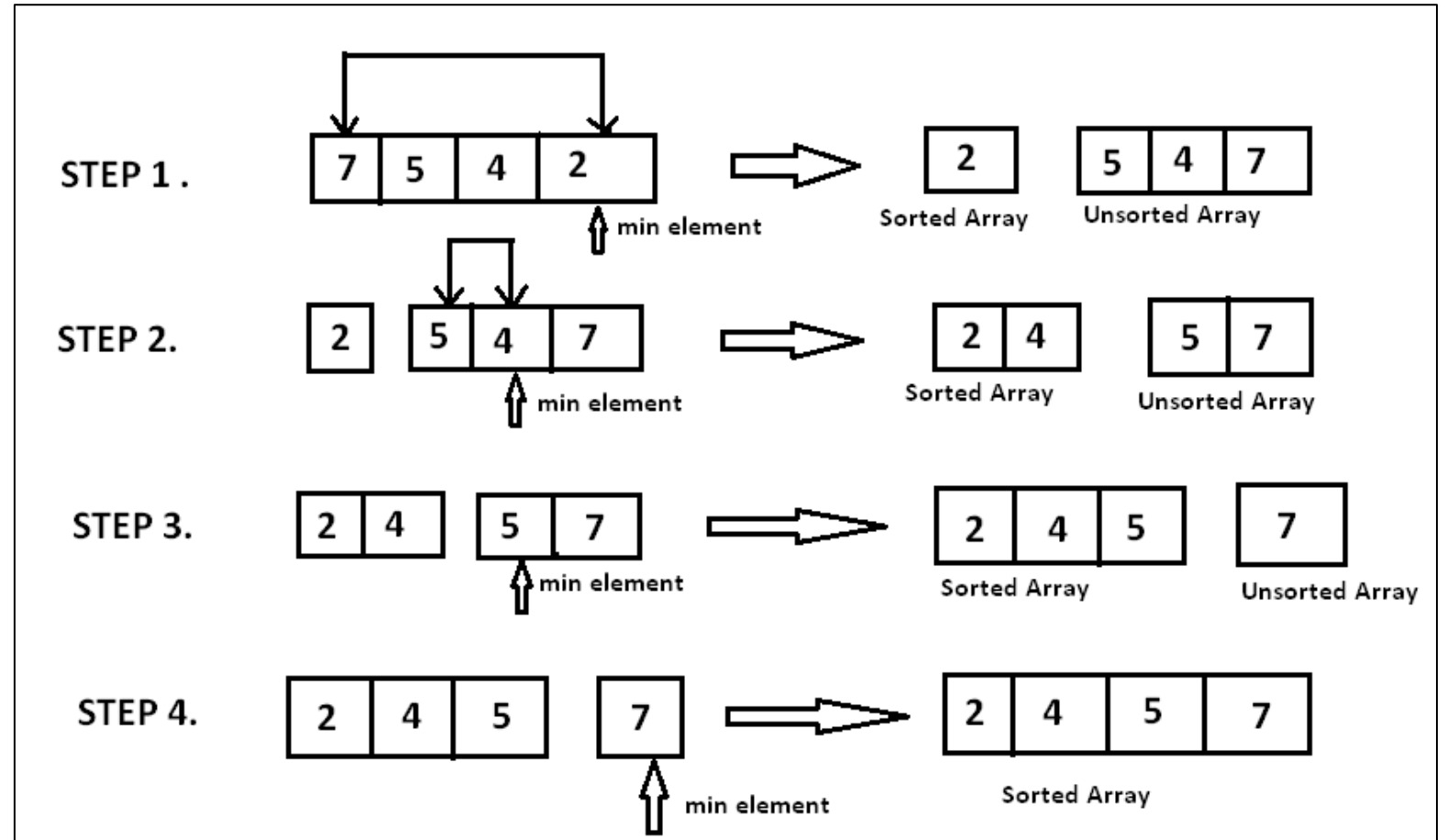
- Algorithme intuitif
- Principe :
 - On cherche le minimum (resp. maximum) dans la collection.
 - On échange ce minimum (resp. maximum) avec le premier élément de la collection.
 - On recommence avec la collection restante, jusqu'à épuisement de tous les items
- Complexité : $O(n^2)$

$$O(n + (n-1) + \dots + 2 + 1) \equiv O\left(\sum_{i=1}^n i\right)$$
$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Tri par sélection (Selection Sort)

Un exemple

<https://www.youtube.com/watch?v=ewU9oET3nJ8>



<https://www.hackerearth.com/practice/algorithms/sorting/selection-sort/tutorial/>

Tri par sélection (Selection Sort)

Implémentation en Python

```
2      # Amine - Colval
3      def selection_sort(values):
4          for i in range(len(values) - 1):
5              min_index = i
6              for j in range(i + 1, len(values)):
7                  if values[j] < values[min_index]:
8                      min_index = j
9              values[i], values[min_index] = values[min_index], values[i]
10
11
12     def main():
13         data = [3, 1, 41, 59, 26, 53, 59, -1]
14         print(data)
15         selection_sort(data)
16         print(data)
```

Tri par insertion (Insertion Sort)

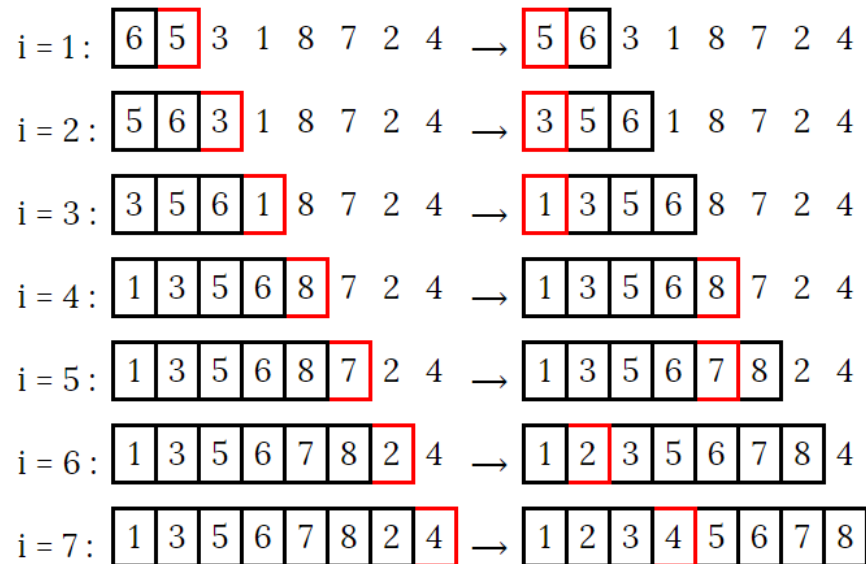
Principe et complexité

- Algorithme intuitif
- Principe :
 - un algorithme qui insère un élément dans une liste d'éléments déjà triés (une file à priorité).
 - On suppose que les i premiers éléments de la liste sont déjà triés.
 - On insère à la bonne place l'élément $(i+1)$ parmi les i premiers de façon à respecter l'ordre déjà établi.
 - On itère jusqu'à insérer le *dernier élément*.
- Complexité : $O(n^2)$
 - Dans le pire des cas, chaque élément est comparé à tous les autres éléments du tableau trié. Pour les éléments, il y aura des comparaisons. Par conséquent, la complexité temporelle est $O(n^2)$.

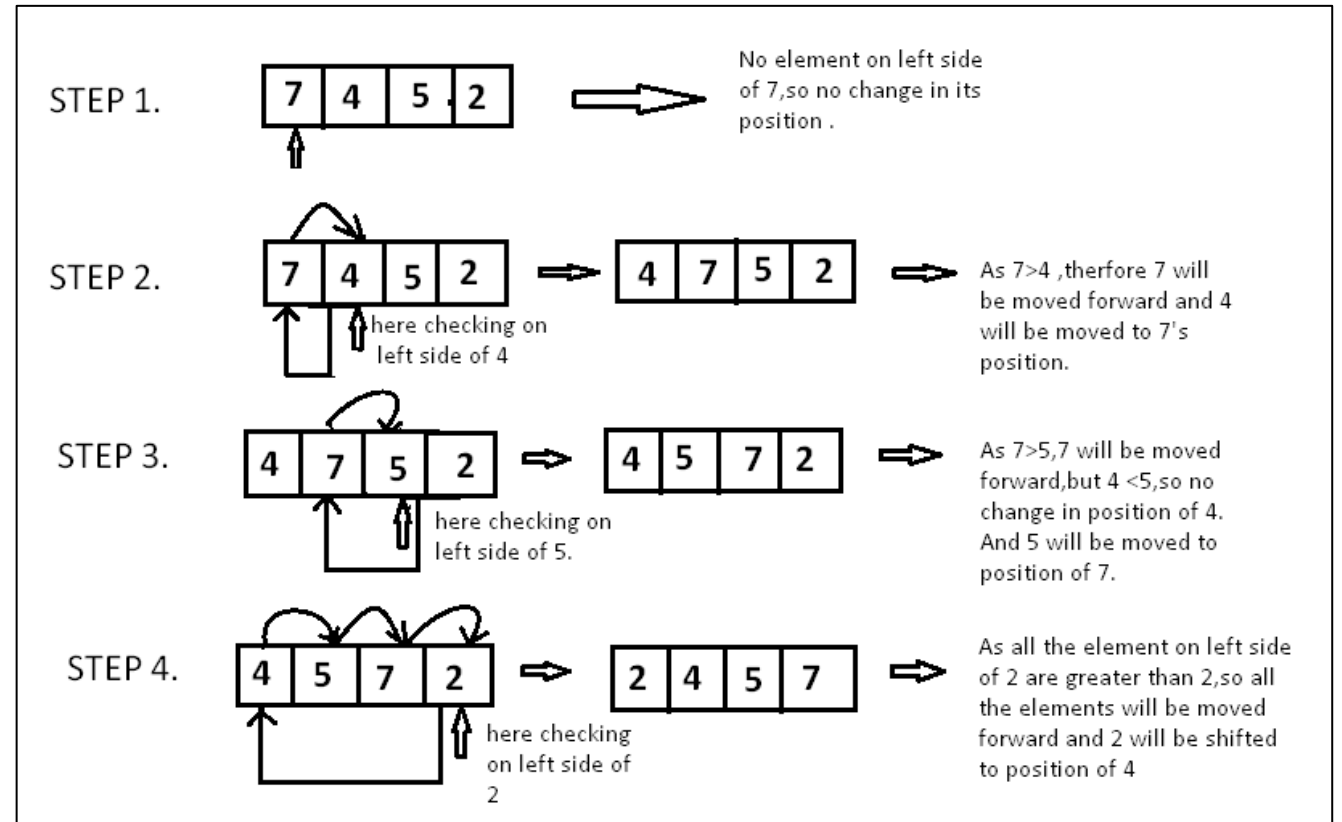
Tri par insertion (Insertion Sort)

Exemples

https://www.youtube.com/watch?v=v_eKybSV4yk



https://www.wikiwand.com/fr/Tri_par_insertion



<https://www.hackerearth.com/practice/algorithms/sorting/insertion-sort/tutorial/>

Tri par insertion (Insertion Sort)

Implémentation en Python

```
2      # Amine - Colval
3      def insertion_sort(values):
4          for index in range(0, len(values)-1):
5              current_position = index
6              value_to_sort = values[index]
7              while current_position > 0 and \
8                  values[current_position - 1] > value_to_sort:
9                  values[current_position] = values[current_position - 1]
10                 current_position -= 1
11                 values[current_position] = value_to_sort
12
13
14     def main():
15         data = [3, 1, 41, 59, 26, 53, 59]
16         print(data)
17         insertion_sort(data)
18         print(data)
```

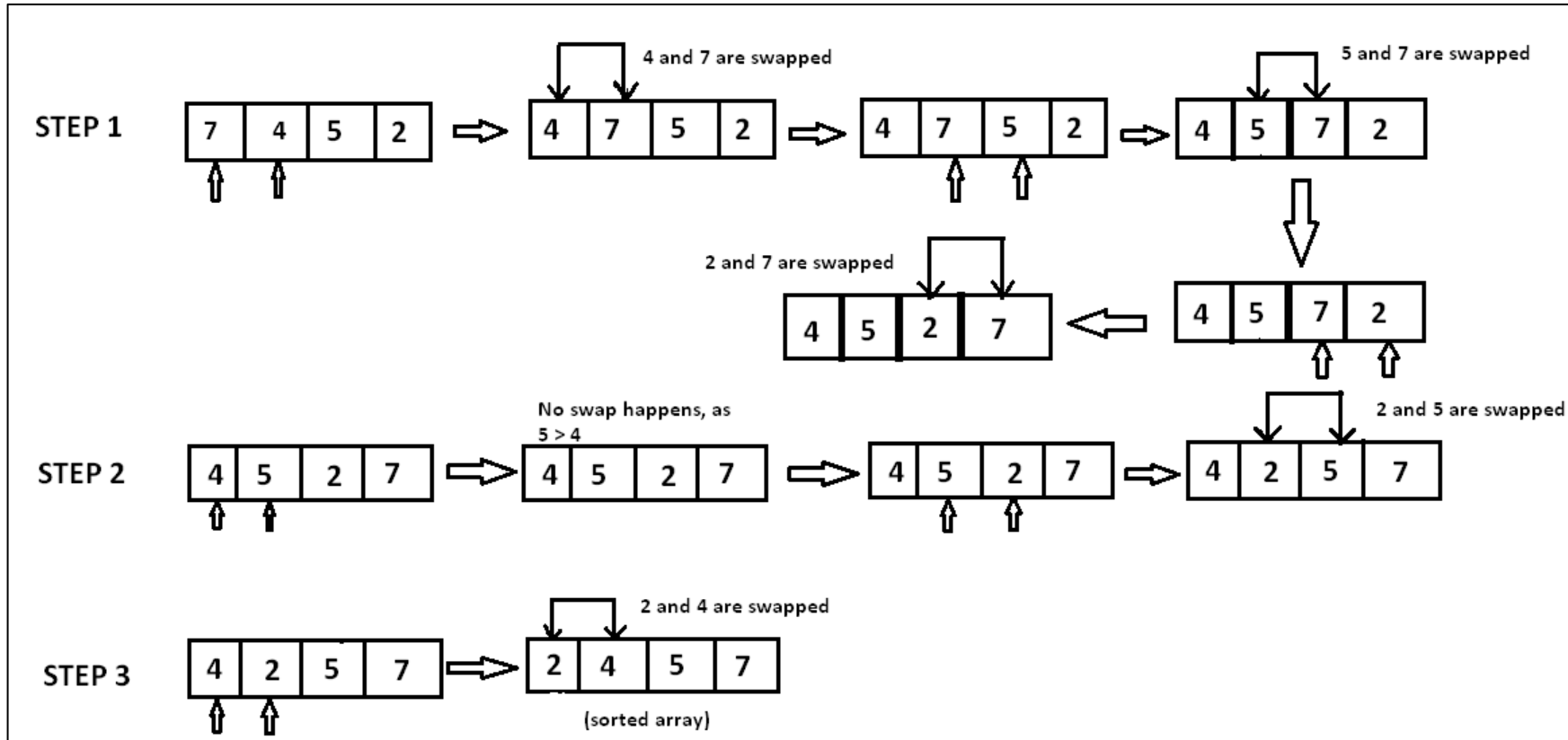
Tri par bulles (Bubble Sort)

Principe et complexité

- Principe :
 - On balaye la collection en échangeant deux éléments consécutifs s'ils sont dans le mauvais ordre. Le plus grand élément de la collection est donc repoussé à la fin.
 - On recommence à partir du début, avec les $n - 1$ premiers éléments et ainsi de suite.
 - Cela ressemble un peu au tri par sélection, à ceci près qu'on utilise la recherche du maximum pour corriger quelques autres inversions.
- Le nom vient de ce que les grands éléments sautent par dessus les plus petits jusqu'à la fin du tableau, comme des bulles
- Complexité : $O(n^2)$
 - The complexity of bubble sort is $O(n^2)$ in worst case, because the entire array needs to be iterated for every element.

Tri par bulles (Bubble Sort) Exemples

<https://www.youtube.com/watch?v=nmhjrl-aW5o>



<https://www.hackerearth.com/practice/algorithms/sorting/bubble-sort/tutorial/>

Tri par bulles (Bubble Sort)

Implémentation en Python

```
2      # Amine - Colval
3      def bubble_sort(values):
4          for i in range(len(values)):
5              for j in range(len(values) - 1):
6                  if values[j] > values[j + 1]:
7                      values[j], values[j + 1] = values[j + 1], values[j]
8
9
10     def bubble_sort_v2(values):
11         has_swapped = True
12         while has_swapped:
13             has_swapped = False
14             for i in range(len(values) - 1):
15                 if values[i] > values[i + 1]:
16                     values[i], values[i + 1] = values[i + 1], values[i]
17             has_swapped = True
```

Tri par fusion (Merge Sort)

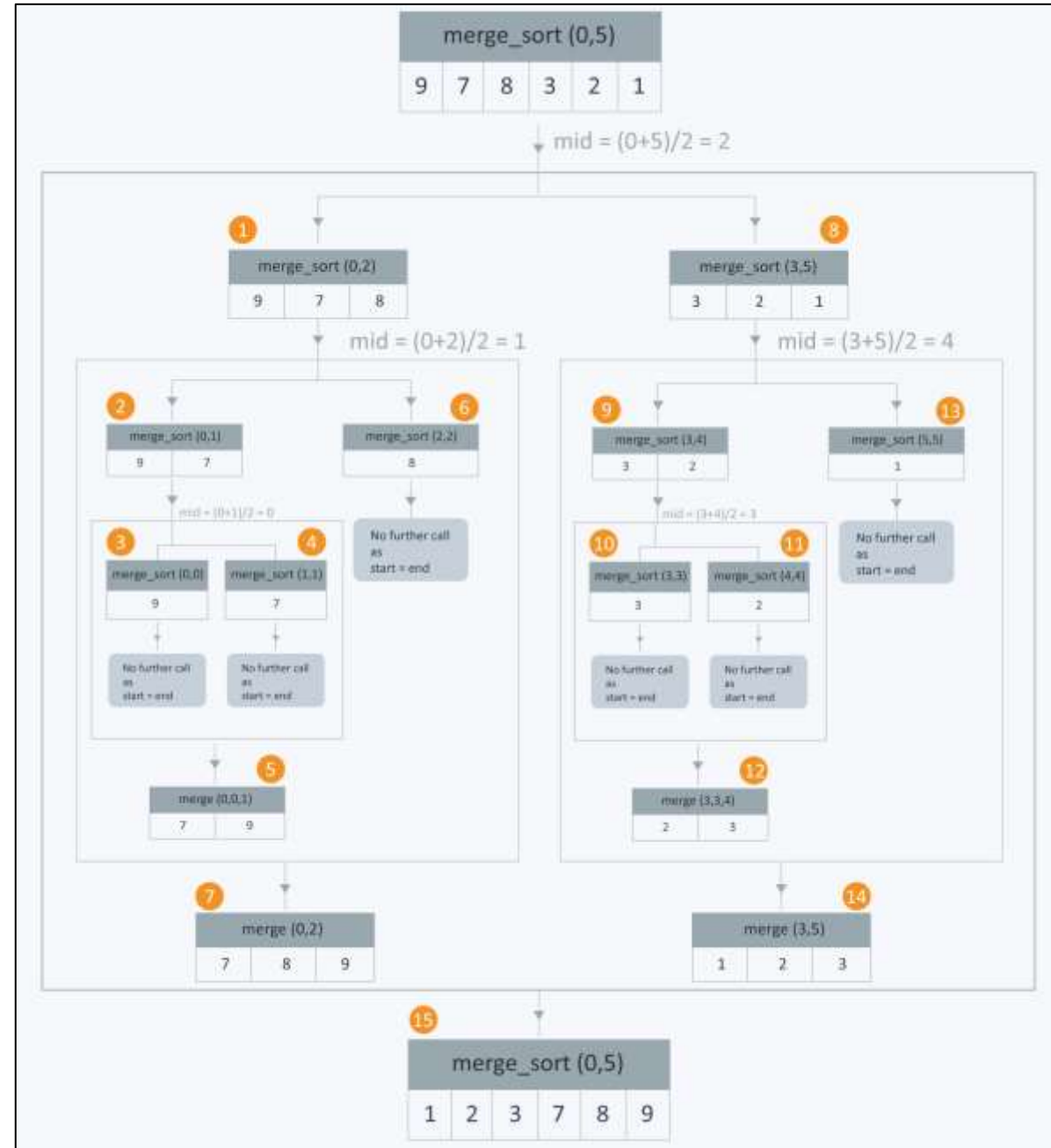
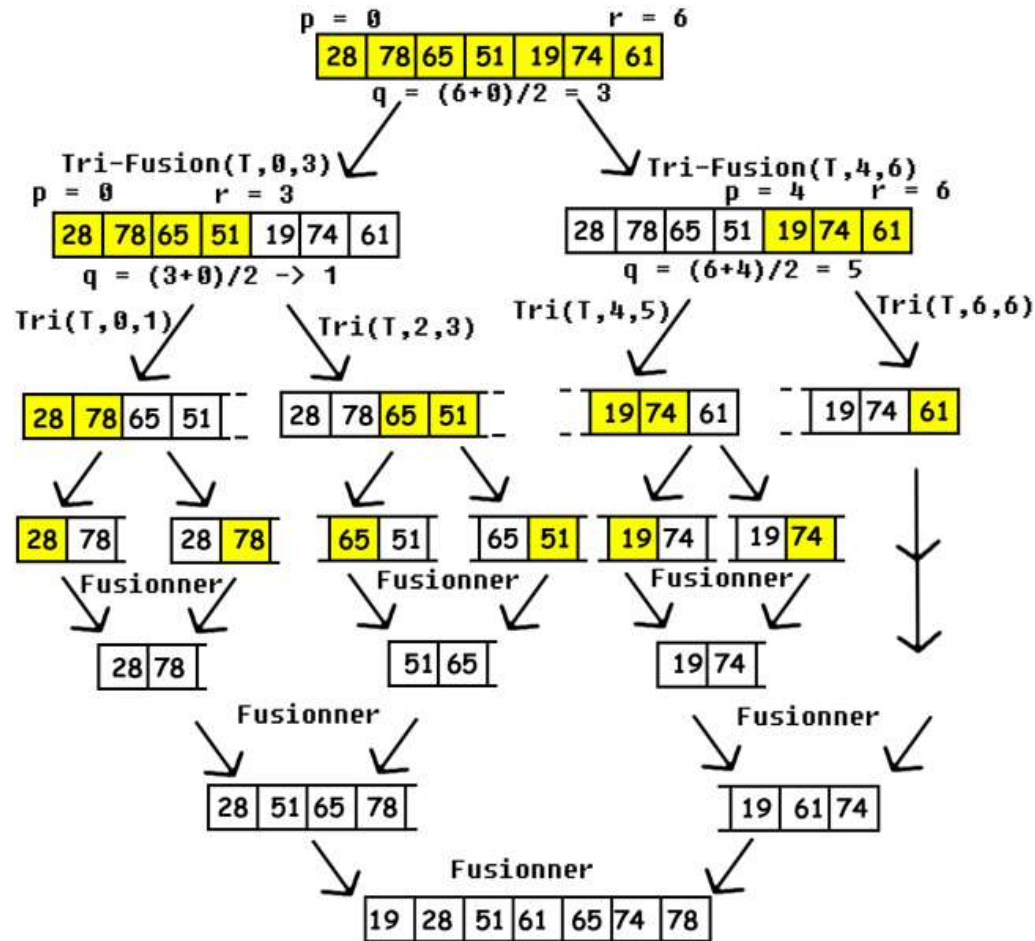
Principe et complexité

- Technique : diviser pour régner
- Principe :
 - On divise la collection en deux parties égales (de taille $n/2$).
 - On trie chacune de ces deux parties.
 - Finalement, on fusionne ces deux parties triées.
- Complexité : $O(n \log n)$
 - La liste de taille n est divisée en un maximum de $(\log n)$ parties, et la fusion de toutes les sous-listes en une seule liste prend un temps $O(n)$, le pire des cas d'exécution de cet algorithme est $O(n \log n)$.

Tri par fusion

Exemples

<https://www.youtube.com/watch?v=MbR1kaCyh7g>



Tri par fusion

Implémentation en Python

```
5 def merge_sort(values, start, end):
6     if start >= end:
7         return
8     middle = (start + end) // 2
9     merge_sort(values, start, middle)
10    merge_sort(values, middle + 1, end)
11    merge(values, start, end, middle)
```

```
14 def merge(values, start, end, middle):
15     left_copy = values[start:middle + 1]
16     right_copy = values[middle + 1:end + 1]
17     left_copy_index = 0
18     right_copy_index = 0
19     sorted_index = start
20
21     while left_copy_index < len(left_copy) and right_copy_index < len(right_copy):
22         if left_copy[left_copy_index] <= right_copy[right_copy_index]:
23             values[sorted_index] = left_copy[left_copy_index]
24             left_copy_index = left_copy_index + 1
25         else:
26             values[sorted_index] = right_copy[right_copy_index]
27             right_copy_index = right_copy_index + 1
28         sorted_index = sorted_index + 1
29
30     while left_copy_index < len(left_copy):
31         values[sorted_index] = left_copy[left_copy_index]
32         left_copy_index = left_copy_index + 1
33         sorted_index = sorted_index + 1
34
35     while right_copy_index < len(right_copy):
36         values[sorted_index] = right_copy[right_copy_index]
37         right_copy_index = right_copy_index + 1
38         sorted_index = sorted_index + 1
```

Tri insertion binaire (Binary insertion sort)

Principe et complexité

- Une variante du tri insertion dont la complexité est $O(n^2)$
 - Au pire des cas, chaque élément est comparé à tous les autres éléments du tableau trié.
- Principe : faire une recherche binaire de la position d'insertion d'un élément dans une liste déjà triés (une file à priorité).
- Complexité:
 - La recherche binaire coûte au pire cas $\log_2(n)$ comparaisons
 - Pour insérer les n éléments, le coût total est $O(n \log_2 n)$
 - Toutefois, l'algorithme présente une complexité quadratique $O(n^2)$ à cause des swaps nécessaires à chaque itération (un décalage des valeurs pour libérer la position d'insertion du nouvel élément est nécessaire). Le nombre total de décalages faits est $n(n-1)/2$.
- Animation : https://www.youtube.com/watch?v=sr_bR1WwcLY

Tri insertion binaire (Binary insertion sort)

Implémentation en Python

```
13 def binary_insertion_sort(values):
14     for i in range(1, len(values)):
15         value = values[i]
16         position = binary_search_recursive(values, value, 0, i - 1)
17         values = values[:position] + [value] + values[position:i] + values[i + 1:]
18     return values
```

```
21 # return position of item to find
22 def binary_search_recursive(items, item_to_find, start, end):
23     if start == end:
24         if items[start] > item_to_find:
25             return start
26         else:
27             return start + 1
28     if start > end:
29         return start
```

```
30
31     middle = (start + end) // 2
32     if item_to_find == items[middle]:
33         return middle
34     if item_to_find < items[middle]:
35         return binary_search_recursive(items, item_to_find, start, middle - 1)
36     else:
37         return binary_search_recursive(items, item_to_find, middle + 1, end)
```

Références

- <https://opensa-server.cs.vt.edu/ODSA/Books/CS3/html/>
- Bubble sort : https://www.youtube.com/watch?v=b0SsPQYKq_U&list=PLLMXbkbDbVt9MIJ9DV4ps-trOzWtphYO&index=166
- <https://gist.github.com/abonander/6648739>
- <https://medium.com/@AlirezaMeskin/sorting-algorithms-in-functional-ways-in-scala-66aa4bf6414e>
- <https://www.youtube.com/watch?v=yUVpaSnzETo>
- https://www.wikiwand.com/fr/Algorithme_de_tri
- <https://github.com/mypetyak/scala-algorithms/tree/master/sort>
- <https://stackabuse.com/selection-sort-in-python/>
- <https://stackabuse.com/merge-sort-in-python/>
- <https://en.wikipedia.org/wiki/Timsort>
- <https://medium.com/@rscheiwe/the-case-for-timsort-349d5ce1e414>
- <https://hg.python.org/cpython/file/tip/Objects/listsort.txt>
- <https://www.chrislaux.com/timsort.html>